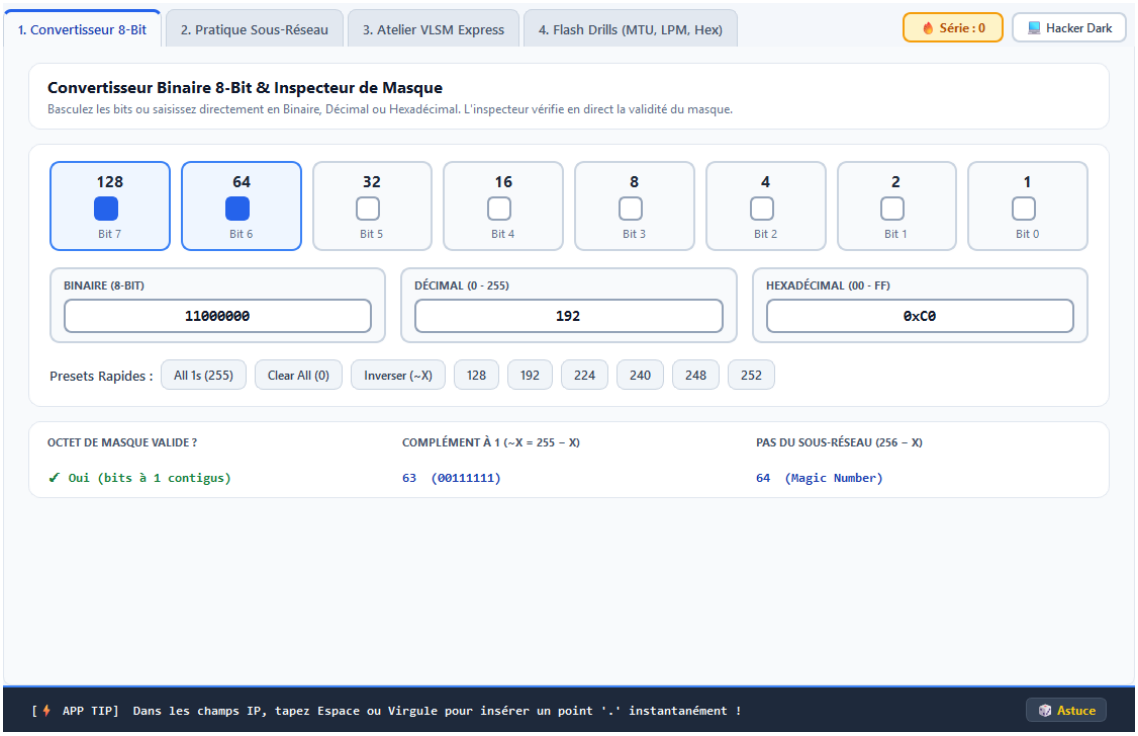
INSTITUT NATIONAL DES POSTES ET TÉLÉCOMMUNICATIONS (INPT)

Cursus Ingénieur — Filière ASEDS (Systèmes Embarqués & Réseaux)

# IPv4 Subnet Toolkit v1.2.0

## Manuel Intégral d'Apprentissage, Mathématiques Réseau, Architecture Logicielle & Guide d'Ingénierie du Code Source

*Recueil exhaustif de la documentation pédagogique et technique : Algèbre booléenne 8/32 bits, découpage VLSM décroissant, solveurs MTU / LPM / Checksum 16 bits, ingénierie d'interface PyQt6 anti-saturation, modèle MVC découplé et blueprint de portage Java 17/21 OOP.*




**Version 1.2.0 — Octobre 2026**

Édition Intégrale dédiée aux Étudiants et Enseignants INPT ASEDS

# Contents


**I Architecture Fondamentale & Mathématiques Réseau** 4

**1 Architecture Globale : Du Monolithe v1.0.0 à la Clean Architecture v1.2.0** 5

1.1 Pourquoi Refactoriser en Deux Paquetages Découplés ? . . . . . 5

1.2 Schéma du Modèle MVC Découplé . . . . . 6

**2 Bitwise Math & Manipulation d'Octets (`engine/ipv4_math.py`)** 7

2.1 Contiguïté des Bits d'un Octet de Masque IPv4 . . . . . 7

2.2 Le Piège du Complément à Un en Python (`~val`) . . . . . 7

2.3 Le "Magic Number" (Taille du Bloc / Pas du Sous-Réseau) . . . . . 8

2.4 Formules Fondamentales 32 Bits du Cours INPT . . . . . 8

2.5 Cas d'Étude 154.4.32.0/19 : Bits Empruntés et d'Hôtes . . . . . 8

2.6 Portées d'Adressage IP (RFC 1918, Loopback & APIPA) . . . . . 9

**II Algorithmes du Cours INPT & Résolveurs Dédiés** 10

**3 Découpage VLSM & Preuve du Tri Décroissant (`engine/vlsm_solver.py`)** 11

3.1 Pourquoi Trier les Besoins par Ordre Strictement Décroissant ? . . . . . 11

3.2 Formule de Dimensionnement d'un Sous-Réseau VLSM . . . . . 11

3.3 Corrigé Intégral des 2 Presets Officiels du Cours INPT . . . . . 11

3.3.1 Preset Cours 1 : Réseau `193.225.34.0/24`, Besoins : `[44, 20, 100]` . . . . . 11

3.3.2 Preset Cours 2 : Réseau `192.168.0.0/24`, Besoins : `[28, 4, 100]` . . . . . 12

**4 Solveurs des Mini-Exercices Avancés (`engine/drills_solver.py`)** 13

4.1 Fragmentation IPv4 & Contraintes MTU (RFC 791) . . . . . 13

4.1.1 Démonstration sur l'Exemple Officiel (`Total = 4000B, IHL = 5, MTU = 1500B`) 13

4.2 Routage IPv4 & Hiérarchie de Décision d'un Routeur . . . . . 14

4.3 Checksum d'En-tête IPv4 sur 16 bits (RFC 791 / RFC 1071) . . . . . 14

4.3.1 Démonstration Pas à Pas sur l'Exemple du Cours . . . . . 14

**III Gamification Pédagogique & Moteur d'Astuces** **16**

**5 Moteur de Progression & Banque d'Astuces (`engine/tips_bank.py`)** **17**

5.1 Philosophie de la Gamification Réseau . . . 17

5.2 Catalogue Complet des 16 Astuces Réseau (`TIPS_LIST`) . . . 17

5.3 Système de Trophées (`ADVANCEMENTS_CATALOG`) . . . 18

**IV Ingénierie Graphique PyQt6 & Ergonomie Anti-Saturation** **19**

**6 Composants Graphiques Sur-Mesure (`ui/widgets.py`)** **20**

6.1 Saisie Assistée d'Adresses IP : `IpLineEdit` . . . 20

6.2 Bannières et Toasts Temporisés par `QTimer` . . . 21

**7 Feuilles de Style QSS & Défi du "Zero Pixel Shift" (`ui/themes.py`)** **22**

7.1 Principes du QSS et Rafraîchissement Dynamique . . . 22

7.2 Garantie du "Zero Pixel Shift" entre Thèmes . . . 22

**V Revue de Code Détaillée des 4 Onglets Spécialisés** **23**

**8 Onglet 1 : Convertisseur Binaire 8-Bit (`ui/tab_binary.py`)** **24**

8.1 Architecture Graphique & Synchronisation Quadridirectionnelle . . . 24

8.2 Implémentation du Verrou Anti-Récursion : `self._updating` . . . 24

**9 Onglet 2 : Pratique du Sous-Réseau & Tiroir Binaire 32-bit (`ui/tab_subnet.py`)** **26**

9.1 Modèle Pédagogique & Divulgation Progressive . . . 26

9.2 Gestion du Tiroir et Rafraîchissement QSS . . . 27

**10 Onglet 3 : Atelier VLSM Express (`ui/tab_vlsm.py`)** **28**

10.1 Ergonomie "Zéro Surcharge Cognitive" & Auto-Complétion . . . 28

10.2 Changement d'Exercice sans Faux Positifs : `blockSignals(True)` . . . 29

**11 Onglet 4 : Flash Drills (`ui/tab_drills.py`)** **30**

11.1 Structure Multi-Exercices à Pagination `QStackedWidget` . . . 30

11.2 Algorithme de Vérification LPM . . . 31

**VI Système, Packaging & Blueprint de Portage Java OOP** **32**

**12 Cycle de Vie de l'Application & Compilation Standalone** **33**

12.1 Point d'Entrée `converter.py` . . . 33

12.2 Compilation Binaire Standalone avec PyInstaller . . . 33

**13 Blueprint de Portage vers Java 17/21 Orienté Objet** **34**

13.1 Correspondance des Modèles de Données (`record` Java) . . . . . 34

13.2 Pièges de l'Arithmétique 32 bits en Java (`int` signé vs `long`) . . . . . 34

13.3 Architecture de Packages Java Recommandée . . . . . 35

**Conclusion & Synthèse Pédagogique** **36**

# Part I

# Architecture Fondamentale & Mathématiques Réseau

# Chapter 1

# Architecture Globale : Du Monolithe v1.0.0 à la Clean Architecture v1.2.0

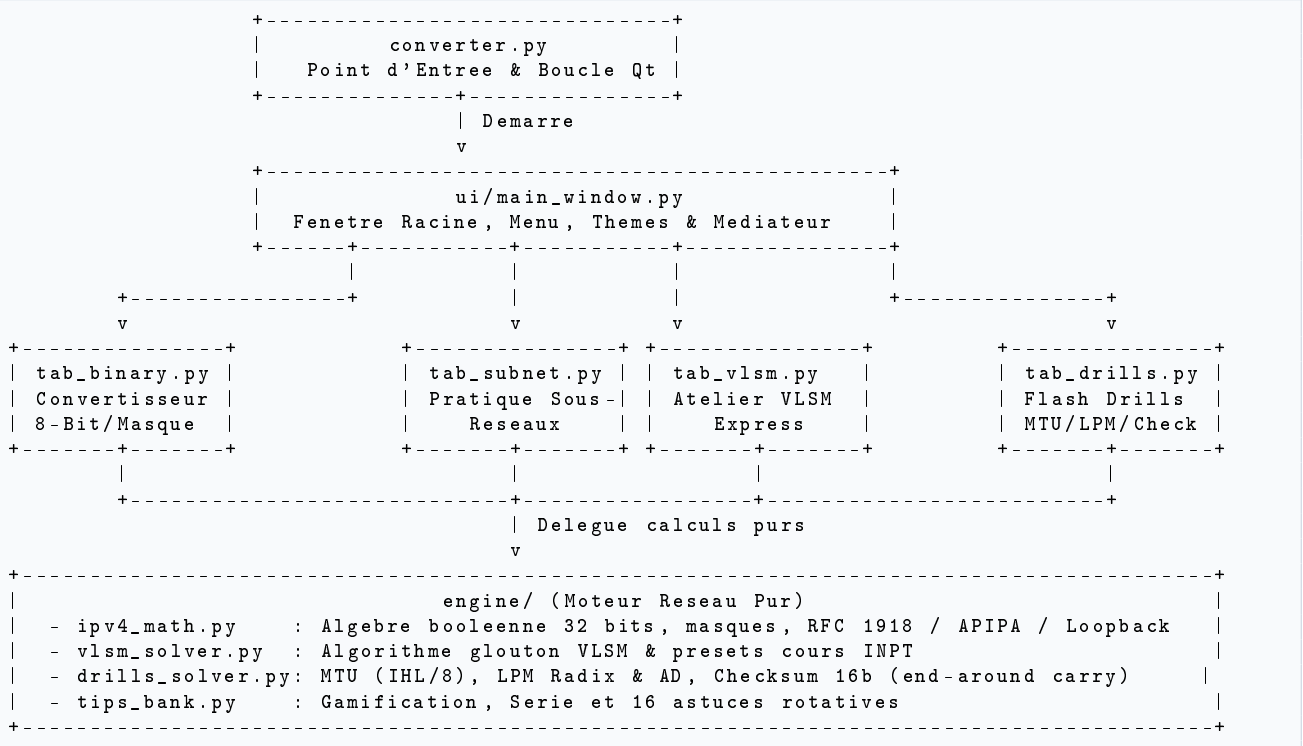
## 1.1 Pourquoi Refactoriser en Deux Paquetages Découplés ?

Dans la version initiale `v1.0.0`, l'intégralité du programme résidait dans un fichier unique `converter.py` (1 171 lignes de code). Les calculs d'adresses IP, l'algorithmique binaire et la validation des saisies étaient imbriqués à l'intérieur des méthodes de classes graphiques `QWidget`.

Pour la version **v1.2.0**, nous appliquons le principe fondamental de **Séparation des Préoccupations (*Separation of Concerns — SoC*)** :

1. **Zéro dépendance GUI dans `engine/`** : Aucun fichier du paquetage moteur n'importe `PyQt6`. Le moteur ne manipule que des types primitifs Python (`int`, `str`, `bool`, `list`) et des objets de transfert immuables (`@dataclass(frozen=True)`).
2. **Testabilité unitaire immédiate** : 100 % des calculs, des exercices VLSM et des cas limites peuvent être exécutés et validés en ligne de commande en quelques millisecondes, sans devoir initialiser de serveur X ou de contexte graphique.
3. **Portabilité vers Java OOP (Cursus ASEDS)** : Chaque `@dataclass` correspond rigoureusement à un `record` Java 16+ ou à une classe POJO finale. Chaque classe de service (`IPv4MathEngine`, `VlsmSolver`, `DrillsSolver`) regroupe des méthodes pures sans état, triviales à traduire en méthodes `public static` en Java.

## 1.2 Schéma du Modèle MVC Découplé




**Règle d'Or de l'Architecture**

Le moteur (`engine/`) est complètement agnostique de l'interface utilisateur (`ui/`). Les formulaires graphiques appellent les méthodes pures de `engine/`, reçoivent des résultats immuables et rafraîchissent leurs labels sans jamais effectuer d'arithmétique interne.

# Chapter 2

# Bitwise Math & Manipulation d'Octets (`engine/ipv4_math.py`)

## 2.1 Contiguïté des Bits d'un Octet de Masque IPv4

Un masque de sous-réseau IPv4 valide est formé d'une séquence continue de bits à 1 (partie réseau), suivie d'une séquence continue de bits à 0 (partie hôte). Sur un seul octet (8 bits), il n'existe donc mathématiquement que **9 valeurs décimales valides** :

| Octet Binaire | Valeur Décimale | Hexadécimal | Nombre de Bits à 1 |
|---|---|---|---|
| `00000000` | 0 | `0x00` | 0 |
| `10000000` | 128 | `0x80` | 1 |
| `11000000` | 192 | `0xC0` | 2 |
| `11100000` | 224 | `0xE0` | 3 |
| `11110000` | 240 | `0xF0` | 4 |
| `11111000` | 248 | `0xF8` | 5 |
| `11111100` | 252 | `0xFC` | 6 |
| `11111110` | 254 | `0xFE` | 7 |
| `11111111` | 255 | `0xFF` | 8 |

Table 2.1: Les 9 octets de masque contigus autorisés en IPv4.

Toute valeur présentant une alternance de 0 et 1 (comme $160 = \texttt{10100000}_2$) est formellement invalide pour un masque.

## 2.2 Le Piège du Complément à Un en Python (`~val`)

En Python, le type `int` manipule des entiers signés à précision arbitraire. L'opérateur unaire `~x` calcule mathématiquement $-x - 1$ (complément à deux signé). Ainsi, `~192` donne `-193`.

Pour obtenir le **complément à un non signé** (inversion bit à bit pure) :

- **Sur 8 bits** : `(~val) & 0xFF`, ce qui équivaut exactement à l'arithmétique :

$$\text{Complément}_8 = 255 - \text{val}$$

- **Sur 32 bits** (Masque générique / Wildcard Mask) :

$$\text{Wildcard} = (\sim \text{Masque}) \ \& \ \text{0xFFFFFFFF}$$

## 2.3 Le "Magic Number" (Taille du Bloc / Pas du Sous-Réseau)

Pour un octet de masque $M \in \{128, 192, 224, 240, 248, 252, 254, 255\}$, le pas de progression entre deux sous-réseaux consécutifs dans cet octet vaut :

$$\text{Magic Number} = 256 - M$$

*Exemple d'application* : Pour le masque `255.255.255.192` (`/26`), le pas dans le 4ème octet est $256 - 192 = 64$. Les sous-réseaux commencent donc à `.0`, `.64`, `.128`, `.192`.

## 2.4 Formules Fondamentales 32 Bits du Cours INPT

Soit IP et CIDR les paramètres d'entrée :

$$\begin{aligned}
\text{Masque 32 bits} &= (\text{0xFFFFFFFF} \ll (32 - \text{CIDR})) \ \& \ \text{0xFFFFFFFF} \\
\text{Wildcard Mask} &= (\sim \text{Masque}) \ \& \ \text{0xFFFFFFFF} \\
\text{Adresse Réseau} &= \text{IP} \ \& \ \text{Masque} \\
\text{Host ID Interne} &= \text{IP} \ \& \ \text{Wildcard Mask} \\
\text{Adresse de Broadcast} &= \text{Réseau} \mid \text{Wildcard Mask} \\
\text{Premier Hôte} &= \text{Réseau} + 1 \quad (\text{si CIDR} \leq 30) \\
\text{Dernier Hôte} &= \text{Broadcast} - 1 \quad (\text{si CIDR} \leq 30) \\
\text{Sous-Réseau Suivant} &= \text{Réseau} + 2^{32-\text{CIDR}}
\end{aligned}$$

## 2.5 Bits Empruntés ($b_S$) et Bits d'Hôtes ($b_H$) : Étude du Cas `154.4.32.0/19`

Dans la théorie classique par classe (RFC 791) :

- **Classe A** : 1er octet de 0 à 127 (bit de tête `0...`), masque par défaut `/8`.
- **Classe B** : 1er octet de 128 à 191 (bits de tête `10...`), masque par défaut `/16`.
- **Classe C** : 1er octet de 192 à 223 (bits de tête `110...`), masque par défaut `/24`.
- **Classe D** : 1er octet de 224 à 239 (bits de tête `1110...`), réservée au Multicast.
- **Classe E** : 1er octet de 240 à 255 (bits de tête `11110...`), réservée expérimentale.

Pour l'adresse de référence du cours INPT `154.4.32.0/19` :

1. Le premier octet 154 est compris entre 128 et 191 $\implies$ **Classe B** ($\text{CIDR}_{\text{défaut}} = 16$).
2. Bits empruntés $b_S = \text{CIDR} - \text{CIDR}_{\text{défaut}} = 19 - 16 = 3$ bits $\implies$ $2^3 = 8$ sous-réseaux créés.
3. Bits d'hôtes restants $b_H = 32 - 19 = 13$ bits $\implies$ $2^{13} - 2 = 8\,190$ machines utilisables.
4. Octet significatif : le 3ème octet ($19 = 8 + 8 + 3$). La valeur de masque sur 3 bits est `11100000` (224).
5. Le pas vaut $256 - 224 = 32$. Comme 32 est un multiple de 32, `154.4.32.0` est bien le début du 2ème sous-réseau.
6. Plage d'adresses hôtes : de `154.4.32.1` à `154.4.63.254`, avec diffusion à `154.4.63.255` et prochain sous-réseau à `154.4.64.0`.

## 2.6 Portées d'Adressage IP (RFC 1918, Loopback & APIPA)

- **Loopback** : `127.0.0.0/8` (RFC 1122).
- **APIPA (Automatic Private IP Addressing)** : `169.254.0.0/16` (RFC 3927).
- **Adresses Privées RFC 1918** :
  - `10.0.0.0/8` (`10.0.0.0` à `10.255.255.255`)
  - `172.16.0.0/12` (`172.16.0.0` à `172.31.255.255`)
  - `192.168.0.0/16` (`192.168.0.0` à `192.168.255.255`)
- **Adresses Publiques** : Toutes les adresses unicast non réservées, routables sur l'Internet mondial.

# Part II

# Algorithmes du Cours INPT & Résolveurs Dédiés

# Chapter 3

# Découpage VLSM & Preuve du Tri Décroissant (`engine/vlsm_solver.py`)

## 3.1 Pourquoi Trier les Besoins par Ordre Strictement Décroissant ?

En VLSM (*Variable Length Subnet Mask*), un bloc de taille $2^h$ doit obligatoirement commencer à une adresse IP dont l'entier 32 bits est un **multiple exact de sa propre taille** $2^h$ (alignement binaire des frontières de sous-réseau).

- En allouant d'abord les plus grands blocs ($2^{h_1} \geq 2^{h_2} \geq \dots$), chaque sous-réseau consécutif démarre naturellement sur un multiple de sa propre taille sans laisser de trou.
- Si l'on allouait un petit bloc en premier (ex: `/27`, bloc de 32 de `.0` à `.31`), le bloc suivant de taille 128 (`/25`) ne pourrait pas commencer à `.32` car 32 n'est pas un multiple de 128. Il devrait sauter à `.128`, gaspillant 96 adresses intermédiaires !

## 3.2 Formule de Dimensionnement d'un Sous-Réseau VLSM

Pour un besoin de $N$ machines hôtes :

1. On cherche le plus petit entier $h \geq 2$ satisfaisant :

$$2^h - 2 \geq N \iff 2^h \geq N + 2 \iff h = \max\left(2, \lceil \log_2(N+2) \rceil\right)$$

   Les 2 adresses soustraites correspondent à l'adresse Réseau (tous les bits hôtes à 0) et à l'adresse de Broadcast (tous les bits hôtes à 1).

2. Le préfixe CIDR optimal vaut : $\text{CIDR} = 32 - h$.
3. La taille du bloc alloué vaut : $\text{BlockSize} = 2^h$.

## 3.3 Corrigé Intégral des 2 Presets Officiels du Cours INPT

### 3.3.1 Preset Cours 1 : Réseau `193.225.34.0/24`, Besoins : `[44, 20, 100]`

1. **Étape 1 — Tri Décroissant Impératif** : 100 hôtes, puis 44 hôtes, puis 20 hôtes.
2. **Sous-Réseau 1 (100 hôtes)** :

$$2^7 - 2 = 126 \geq 100 \implies h = 7 \implies \text{CIDR} = 32 - 7 = 25 \text{ (Masque 255.255.255.128)}$$

Taille du bloc : $2^7 = 128$.

- Adresse Réseau : `193.225.34.0/25`
- Plage d'hôtes utilisables : `193.225.34.1` à `193.225.34.126`
- Adresse de Broadcast : `193.225.34.127`

3. **Sous-Réseau 2 (44 hôtes)** : Démarre à l'adresse consécutive `193.225.34.128`.

$$2^6 - 2 = 62 \geq 44 \implies h = 6 \implies \text{CIDR} = 32 - 6 = 26 \text{ (Masque 255.255.255.192)}$$

Taille du bloc : $2^6 = 64$.

- Adresse Réseau : `193.225.34.128/26`
- Plage d'hôtes utilisables : `193.225.34.129` à `193.225.34.190`
- Adresse de Broadcast : `193.225.34.191`

4. **Sous-Réseau 3 (20 hôtes)** : Démarre à l'adresse consécutive `193.225.34.192`.

$$2^5 - 2 = 30 \geq 20 \implies h = 5 \implies \text{CIDR} = 32 - 5 = 27 \text{ (Masque 255.255.255.224)}$$

Taille du bloc : $2^5 = 32$.

- Adresse Réseau : `193.225.34.192/27`
- Plage d'hôtes utilisables : `193.225.34.193` à `193.225.34.222`
- Adresse de Broadcast : `193.225.34.223`

Capacité restante disponible : de `.224` à `.255` (bloc de 32 adresses libres).

### 3.3.2 Preset Cours 2 : Réseau `192.168.0.0/24`, Besoins : `[28, 4, 100]`

1. Tri : 100, puis 28, puis 4.
2. **SR 1 (100 hôtes)** : $h = 7 \implies$ `/25` (bloc de 128) $\implies$ `192.168.0.0/25`, hôtes `.1` à `.126`, broadcast `.127`.
3. **SR 2 (28 hôtes)** : $2^5 - 2 = 30 \geq 28 \implies h = 5 \implies$ `/27` (bloc de 32) $\implies$ `192.168.0.128/27`, hôtes `.129` à `.158`, broadcast `.159`.
4. **SR 3 (4 hôtes)** : $2^3 - 2 = 6 \geq 4 \implies h = 3 \implies$ `/29` (bloc de 8) $\implies$ `192.168.0.160/29`, hôtes `.161` à `.166`, broadcast `.167`.

# Chapter 4

# Solveurs des Mini-Exercices Avancés (`engine/drills_solver.py`)

## 4.1 Fragmentation IPv4 & Contraintes MTU (RFC 791)

Lorsqu'un paquet IP traverse un réseau dont la **MTU (Maximum Transmission Unit)** est inférieure à sa taille totale :

1. **Longueur de l'en-tête IP** : Calculée à partir du champ IHL (*Internet Header Length*) exprimé en mots de 32 bits (4 octets) :

$$L_{\text{header}} = \text{IHL} \times 4 \text{ octets} \quad (\text{soit } 20 \text{ octets pour IHL} = 5)$$

2. **Règle impérative du Multiple de 8** : Le champ *Fragment Offset* de l'en-tête IPv4 dispose de 13 bits pour couvrir jusqu'à 65 535 octets ($2^{16}/2^{13} = 8$). L'offset s'exprime donc en **blocs de 8 octets**. La charge utile de chaque fragment (sauf le dernier) doit être un multiple strict de 8 :

$$D_{\max} = \left\lfloor \frac{\text{MTU} - L_{\text{header}}}{8} \right\rfloor \times 8$$

3. **Drapeau MF (More Fragments)** : Vaut 1 si des fragments suivent, 0 pour le dernier.
4. **Champ Offset** : Offset = Position du premier octet de données/8.

### 4.1.1 Démonstration sur l'Exemple Officiel (`Total = 4000B, IHL = 5, MTU = 1500B`)

- Données utiles totales : $4000 - 20 = 3980$ octets.
- Charge utile maximale : $\lfloor (1500 - 20)/8 \rfloor \times 8 = \lfloor 1480/8 \rfloor \times 8 = 1480$ octets.
- **Fragment 1** : Taille totale 1500B (Payload 1480B, octets 0 à 1479), Offset = $0/8 = 0$, MF = 1.
- **Fragment 2** : Taille totale 1500B (Payload 1480B, octets 1480 à 2959), Offset = $1480/8 = 185$, MF = 1.
- **Fragment 3** : Taille totale 1040B (Payload restant 1020B, octets 2960 à 3979), Offset = $2960/8 = 370$, MF = 0.

## 4.2 Routage IPv4 & Hiérarchie de Décision d'un Routeur

Lorsqu'un routeur IP choisit l'interface de sortie d'un paquet, il évalue la table de routage selon un ordre de priorité immuable :

1. **Filtrage d'appartenance** : Seules les routes dont le réseau englobe l'adresse de destination sont conservées.
2. **Règle 1 — Longest Prefix Match (LPM)** : Parmi les routes correspondantes, la route avec le plus grand préfixe CIDR (masque le plus spécifique) gagne toujours, peu importe la distance administrative ou le protocole ! (Exemple : une route `/25` apprise par RIP bat toujours une route `/24` statique !).
3. **Règle 2 — Plus Petite Distance Administrative (AD)** : En cas d'égalité stricte de masque :
   - Connecté (`C`) : AD = 0
   - Statique (`S`) : AD = 1
   - OSPF (`O`) : AD = 110
   - RIP (`R`) : AD = 120
4. **Règle 3 — Plus Faible Métrique** : En cas d'égalité de CIDR et d'AD (même protocole de routage dynamique), le routeur sélectionne la route de plus faible coût (coût OSPF ou nombre de sauts RIP).
5. **Route par Défaut (`0.0.0.0/0`, `S*`)** : Utilisée comme passerelle de dernier recours si aucune route spécifique ne correspond.

## 4.3 Checksum d'En-tête IPv4 sur 16 bits (RFC 791 / RFC 1071)

1. Initialisation : Le champ Checksum de l'en-tête est fixé à `0x0000`.
2. L'en-tête de 20 octets est découpé en **10 mots de 16 bits** hexadécimaux.
3. On calcule la somme brute non signée : $S_{\text{raw}} = \sum_{k=1}^{10} W_k$.
4. **Rabattement des Retenues (*End-Around Carry*)** : Tout dépassement au-delà de 16 bits ($> \texttt{0xFFFF}$) représente une retenue réintroduite en l'additionnant aux bits de poids faible :

```
while carried > 0xFFFF:
    carried = (carried & 0xFFFF) + (carried >> 16)
```

5. **Complément à un Final** : Inversion logique des 16 bits :

$$\text{Checksum} = (\sim S_{\text{carried}}) \ \& \ \text{0xFFFF} = \text{0xFFFF} - S_{\text{carried}}$$

### 4.3.1 Démonstration Pas à Pas sur l'Exemple du Cours

Mots d'en-tête : `4500 0054 0000 4000 4001 0000 C0A8 0068 C0A8 0001`.

- **Étape 1 (Somme brute)** :

$$\begin{aligned} S_{\text{raw}} = {} & \text{0x4500} + \text{0x0054} + \text{0x0000} + \text{0x4000} + \text{0x4001} \\ & + \text{0x0000} + \text{0xC0A8} + \text{0x0068} + \text{0xC0A8} + \text{0x0001} = \text{0x2470E} \end{aligned}$$

- **Étape 2 (End-Around Carry)** : La retenue vaut $0\text{x}2470\text{E} \gg 16 = 0\text{x}2$. Les 16 bits de poids faible valent `0x470E`.

$$S_{\text{carried}} = 0\text{x}470\text{E} + 0\text{x}2 = 0\text{x}4710$$

- **Étape 3 (Complément à un)** :

$$\text{Checksum} = 0\text{xFFFF} - 0\text{x}4710 = 0\text{xB8EF}$$

# Part III

# Gamification Pédagogique & Moteur d'Astuces

# Chapter 5

# Moteur de Progression & Banque d'Astuces (`engine/tips_bank.py`)

## 5.1 Philosophie de la Gamification Réseau

Plutôt que de confronter l'étudiant à un formulaire d'interrogation statique, l'application intègre deux leviers pédagogiques inspirés des jeux vidéo modernes :

1. **Pédagogie Passive (Bannière Tournante)** : Une astuce du cours ou un raccourci clavier défile en continu toutes les 12 secondes, renforçant la mémorisation à long terme sans effort conscient.
2. **Renforcement Positif (Compteur de Flammes & Trophées)** : Résoudre un exercice sans utiliser le bouton d'aide *"Show"* incrémente la série (`Serie : N`) et déclenche une notification dorée de succès débloqué (*Advancement Toast*).

## 5.2 Catalogue Complet des 16 Astuces Réseau (`TIPS_LIST`)

1. **[APP TIP]** : Dans les champs IP, tapez `Espace` ou `Virgule` pour insérer un point `'.'` instantanément !
2. **[APP TIP]** : Smart Auto-Dot v1.2 : dès que vous tapez 2 chiffres $\geq 26$ (ex: 32, 99, 68), le point `'.'` s'ajoute tout seul !
3. **[COURS INPT]** : Les 9 seules valeurs valides d'un octet de masque sont : 255, 254, 252, 248, 240, 224, 192, 128 et 0 (bits à 1 contigus).
4. **[COURS INPT]** : Règle d'Or VLSM : triez TOUJOURS les besoins en machines du plus grand au plus petit avant de découper !
5. **[ASTUCE PRO]** : Magic Number (Pas du sous-réseau) = 256 − Octet du masque. Ex: masque 255.255.255.192 $\rightarrow$ saut de $256 - 192 = 64$.
6. **[COURS INPT]** : En VLSM, n'oubliez jamais le +2 : pour $N$ machines, il faut $2^h - 2 \geq N$ (1 adresse Réseau + 1 adresse Broadcast).
7. **[COURS INPT]** : Fragmentation IPv4 : la charge utile (sauf dernier fragment) DOIT être un multiple de 8 octets ! Offset = Octet_Début/8.
8. **[COURS INPT]** : Longueur de l'en-tête IPv4 = IHL $\times$ 4 octets. Avec IHL = 5 (standard sans options), l'en-tête mesure 20 octets.

9. **[COURS INPT]** : Priorité du routeur : 1) Préfixe le plus long (LPM) → 2) Distance Administrative (AD) la plus basse → 3) Métrique la plus faible.
10. **[COURS INPT]** : Distances Administratives (AD) par défaut : Connecté = 0 | Statique = 1 | OSPF = 110 | RIP = 120.
11. **[COURS INPT]** : Plages Privées RFC 1918 : `10.0.0.0/8` (Classe A), `172.16.0.0` à `172.31.255.255` (`/12`, Classe B), `192.168.0.0/16` (Classe C).
12. **[COURS INPT]** : APIPA (`169.254.0.0/16`) est auto-attribuée quand aucun serveur DHCP ne répond. Loopback = `127.0.0.0/8`.
13. **[COURS INPT]** : Classes IPv4 (1er octet) : A [0..127] (`0...`), B [128..191] (`10...`), C [192..223] (`110...`), D [224..239] Multicast, E [240..255] Réservé.
14. **[ASTUCE PRO]** : Host ID Interne = IP & (∼ Masque). L'adresse réseau s'obtient par IP & Masque.
15. **[COURS INPT]** : Checksum IPv4 (16 bits) : additionnez les mots de 16 bits, rabattez la retenue (*end-around carry*) à droite, puis inversez les bits (∼ Somme).
16. **[APP TIP]** : Validez un exercice complet sans cliquer sur *'Show'* pour augmenter votre Série et débloquer des succès !

## 5.3 Système de Trophées (`ADVANCEMENTS_CATALOG`)

- **"Maître du Masque"** (`subnet_clean`) : Sous-réseau résolu sans utiliser 'Show' !
- **"Architecte VLSM"** (`vlsm_clean`) : Découpage VLSM sans gaspillage d'adresses !
- **"Chirurgien MTU"** (`mtu_clean`) : Fragmentation IPv4 découpée au multiple de 8 près !
- **"Routeur Humain"** (`lpm_clean`) : Table de routage et règle LPM/AD maîtrisées !
- **"Seigneur du Checksum"** (`checksum_clean`) : Somme hexa 16 bits et complément à un parfaits !
- **"Série de 3 (On Fire)"** (`streak_3`) : 3 exercices consécutifs validés sans aide !
- **"Ingénieur Réseau INPT"** (`streak_5`) : 5 exercices consécutifs validés sans aide !

# Part IV

# Ingénierie Graphique PyQt6 & Ergonomie Anti-Saturation

# Chapter 6

# Composants Graphiques Sur-Mesure (`ui/widgets.py`)

## 6.1 Saisie Assistée d'Adresses IP : `IpLineEdit`

La saisie répétée d'adresses IPv4 dans un `QLineEdit` standard est une source constante de friction. `IpLineEdit` surcharge `keyPressEvent` :

```
class IpLineEdit(QLineEdit):
    def keyPressEvent(self, event):
        key = event.key()
        text = self.text()
        pos = self.cursorPosition()

        # 1. Remplacement d'Espace ou Virgule par un point
        if key in (Qt.Key.Key_Space, Qt.Key.Key_Comma):
            if pos > 0 and text[pos - 1] not in ". ":
                if text.count(".") < 3:
                    self.insert(".")
                    return
            elif pos < len(text) and text[pos] == ".":
                self.setCursorPosition(pos + 1)
                return
            return

        super().keyPressEvent(event)

        # 2. Algorithme Smart Auto-Dot (v1.2.0)
        if event.text().isdigit():
            cur_text = self.text()
            if self.cursorPosition() == len(cur_text):
                sep = max(cur_text.rfind("."), cur_text.rfind(" "))
                octet = cur_text[sep + 1:] if sep != -1 else cur_text
                if (cur_text.count(".") + cur_text.count(" ")) < 3 and octet.isdigit
                    ():
                    val = int(octet)
                    if len(octet) == 3 or (len(octet) == 2 and val >= 26):
                        self.insert(".")
```

**Explication Mathématique de la Règle `val >= 26`**

Un octet IPv4 ne peut pas dépasser 255. Si l'utilisateur saisit deux chiffres supérieurs ou égaux à 26 (ex: $26, 32, 99$), il est mathématiquement impossible d'ajouter un 3ème chiffre sans dépasser 255 ($260 > 255$). Le composant insère donc le point immédiatement après le 2ème chiffre !

## 6.2 Bannières et Toasts Temporisés par `QTimer`

Pour éviter de figer l'interface avec des appels bloquants (`time.sleep()`) :

- `MinecraftTipBanner` utilise un minuteur périodique (`setInterval(12000)`) pour faire tourner les conseils de façon fluide.
- `AdvancementToast` utilise un minuteur à coup unique (`setSingleShot(True)`, 3200 ms) pour afficher la notification dorée puis la masquer automatiquement sans interruption du flux de travail.

# Chapter 7

# Feuilles de Style QSS & Défi du "Zero Pixel Shift" (ui/themes.py)

## 7.1 Principes du QSS et Rafraîchissement Dynamique

Pour styliser les composants sans dépendre de couleurs codées en dur dans le code Python, nous utilisons les propriétés dynamiques Qt (`setProperty`) combinées aux sélecteurs d'attributs QSS :

```
widget.setProperty("active", True)
widget.style().unpolish(widget)
widget.style().polish(widget)
```

Sans l'appel conjoint à `unpolish()` puis `polish()`, Qt n'actualise pas le rendu graphique pour des motifs d'optimisation interne.

## 7.2 Garantie du "Zero Pixel Shift" entre Thèmes

Lors de la bascule entre le mode clair et le mode sombre Hacker Console, la géométrie globale de la fenêtre reste rigoureusement identique au pixel près :

| Caractéristique | Modern Light Mode | Hacker Console Dark Mode |
|---|---|---|
| Police d'interface | `'Segoe UI', sans-serif` | `'Segoe UI', sans-serif` |
| Police technique IP | `'Consolas', monospace` | `'Consolas', monospace` |
| Taille / Graisse | `13px / 600` | `13px / 600` |
| Padding d'onglet | `8px 14px` | `8px 14px` |
| Bordure structurelle | `1.5px solid #E2E8F0` | `1.5px solid #00551A` |
| Bouton bascule | Largeur fixe 135px | Largeur fixe 135px |

Table 7.1: Paramètres unifiés garantissant zéro décalage de layout.

# Part V

# Revue de Code Détaillée des 4 Onglets Spécialisés

# Chapter 8

# Onglet 1 : Convertisseur Binaire 8-Bit (`ui/tab_binary.py`)

## 8.1 Architecture Graphique & Synchronisation Quadridirectionnelle

L'Onglet 1 (`BinaryConverterTab`) constitue la fondation de l'application. Il organise la manipulation d'un octet IPv4 (0 à 255) autour de 4 zones complémentaires :

1. **Cartes de Bits Interactives** : 8 conteneurs carrés (`#bitBox`) pour les poids $2^7(128)$ à $2^0(1)$.
2. **Trio de Saisie Directe** : Synchronisation simultanée entre Binaire (8 bits), Décimal (0..255) et Hexadécimal (`0x00..0xFF`).
3. **Barre d'Outils Rapide** : Presets pour *All 1s* (255), *Clear* (0), Inversion ($\sim X$) et octets usuels de masques ($128, 192, 224, 240, 248, 252$).
4. **Inspecteur de Masque & Pas** : Détection immédiate de contiguïté des bits à 1, complément à un et calcul du Magic Number.

## 8.2 Implémentation du Verrou Anti-Récursion : `self._updating`

Lorsqu'un utilisateur modifie le champ décimal, celui-ci déclenche la mise à jour des cases à cocher et des champs binaire et hexadécimal. Pour prévenir une boucle d'événements infinie :

```
def apply_value(self, val: int, source: str = ""):
    if self._updating:
        return
    self._updating = True
    try:
        val = max(0, min(255, val))
        self.current_value = val

        # 1. Mise a jour des 8 checkboxes sans declencher on_checkbox_toggled
        for i, cb in enumerate(self.bit_checkboxes):
            bit = (val >> (7 - i)) & 1
            cb.setChecked(bool(bit))

        # 2. Mise a jour des champs texte autres que la source active
        if source != "bin":
            self.bin_input.setText(f"{val:08b}")
        if source != "dec":
            self.dec_input.setText(str(val))
        if source != "hex":
```




Figure 8.1: Onglet 1 en Mode Hacker Dark — Détection en direct d'un octet de masque non contigu invalide (`160 = 10100000`$_2$).

```
            self.hex_input.setText(f"0x{val:02X}")

        # 3. Delegation a engine/ pour l'analyse pure
        analysis = IPv4MathEngine.analyze_octet(val)
        self.update_inspector_labels(analysis)
    finally:
        self._updating = False
```

# Chapter 9

# Onglet 2 : Pratique du Sous-Réseau & Tiroir Binaire 32-bit (`ui/tab_subnet.py`)

## 9.1 Modèle Pédagogique & Divulgation Progressive

L'Onglet 2 (`SubnetPracticeTab`) est dédié à l'entraînement intensif au calcul de sous-réseaux IPv4 standard :

- **Zone de Classification Rapide** : Choix de la classe (`A`, `B`, `C`) et de la portée (`Publique`, `Privée RFC 1918`, `APIPA`, `Loopback`) avec feedback instantané.
- **Tableau des 5 Réponses Clés** : Network, First Host, Last Host, Broadcast, Next Subnet.
- **Tiroir Repliable 32 Bits** : Décomposition booléenne pas à pas (`IP ET Masque`, `Host ID Interne`, bits d'emprunt $b_S$ et bits d'hôtes $b_H$).




Figure 9.1: Onglet 2 en Mode Hacker Dark — Résolution de l'exercice avec déploiement du tiroir binaire 32 bits.

## 9.2 Gestion du Tiroir et Rafraîchissement QSS

```
def toggle_binary_drawer(self):
    """Bascule d'affichage du tiroir independamment de l'onglet actif."""
    should_show = self.drawer_card.isHidden()
    self.drawer_card.setVisible(should_show)
    self.btn_toggle_drawer.setText(
        "Masquer le detail binaire 32 bits" if should_show
        else "Afficher le detail binaire 32 bits"
    )

def set_badge_state(self, badge: QLabel, solved: bool):
    """Application du pattern unpolish/polish pour reactualiser le style QSS."""
    badge.setProperty("solved", "true" if solved else "false")
    badge.style().unpolish(badge)
    badge.style().polish(badge)
```

# Chapter 10

# Onglet 3 : Atelier VLSM Express (`ui/tab_vlsm.py`)

## 10.1 Ergonomie "Zéro Surcharge Cognitive" & Auto-Complétion

L'Atelier VLSM (`VlsmTab`) applique le principe : **l'étudiant prend les décisions d'architecture, l'outil s'occupe des déductions mécaniques**.

- L'étudiant ordonne les besoins par ordre décroissant (*Tri impératif*), déduit le préfixe optimal `/CIDR` ($2^h - 2 \geq N$), et calcule l'**Adresse Réseau**.
- Dès que ces 3 champs sont corrects, le badge latéral d'auto-complétion s'allume en vert et renseigne immédiatement la plage d'hôtes utilisables (`.1` à `.126`), le broadcast et le masque.

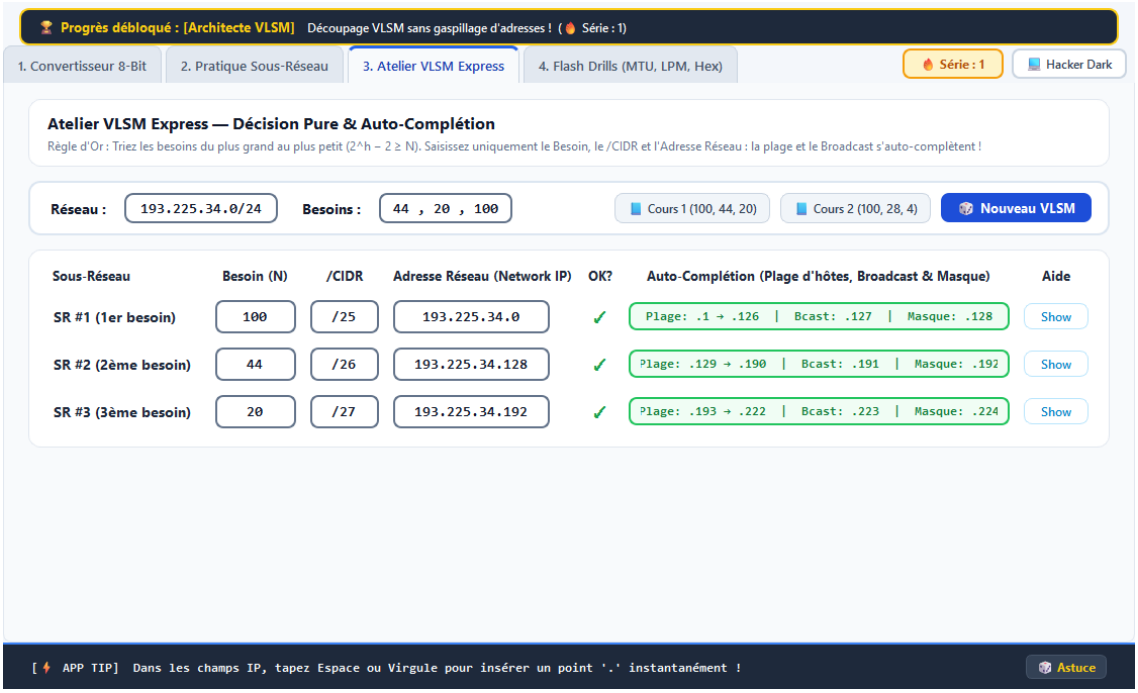



Figure 10.1: Onglet 3 en Mode Modern Light — Exercice Officiel du Cours 1 validé avec auto-complétion des badges.

## 10.2 Changement d'Exercice sans Faux Positifs : `blockSignals(True)`

Lorsqu'un nouveau preset est chargé (ex: Presets officiels du cours 1 et 2), les champs de saisie sont réinitialisés. Pour éviter que les signaux de texte modifié ne déclenchent intempestivement la validation :

```
def load_problem(self, problem: VlsmProblem):
    self.current_problem = problem
    self.base_label.setText(f"Reseau Source : {problem.base_network_str}/{problem.
        base_cidr}")

    for row in self.rows:
        row.need_input.blockSignals(True)
        row.cidr_input.blockSignals(True)
        row.net_input.blockSignals(True)
        try:
            row.need_input.clear()
            row.cidr_input.clear()
            row.net_input.clear()
            row.summary_badge.setText("En attente de validation...")
            row.summary_badge.setProperty("solved", "false")
        finally:
            row.need_input.blockSignals(False)
            row.cidr_input.blockSignals(False)
            row.net_input.blockSignals(False)
```

# Chapter 11

# Onglet 4 : Flash Drills (`ui/tab_drills.py`)

## 11.1 Structure Multi-Exercices à Pagination `QStackedWidget`

L'Onglet 4 (`DrillsTab`) regroupe trois sous-ateliers complémentaires pilotés par un `QStackedWidget` :

1. **Fragmentation MTU** : Calcul de la taille d'en-tête (IHL $\times$ 4), découpage de la charge utile au multiple de 8 près, offset et indicateur MF.
2. **Routage & Longest Prefix Match (LPM)** : Analyse d'une table à 4 routes, détermination de la route gagnante et identification de la règle décisive (*LPM*, *Distance Administrative*, *Métrique*, ou *Route par Défaut*).
3. **Checksum IPv4 (16 bits)** : Somme des 10 mots hexadécimaux, retenue circulaire (*end-around carry*) et complément à un final.

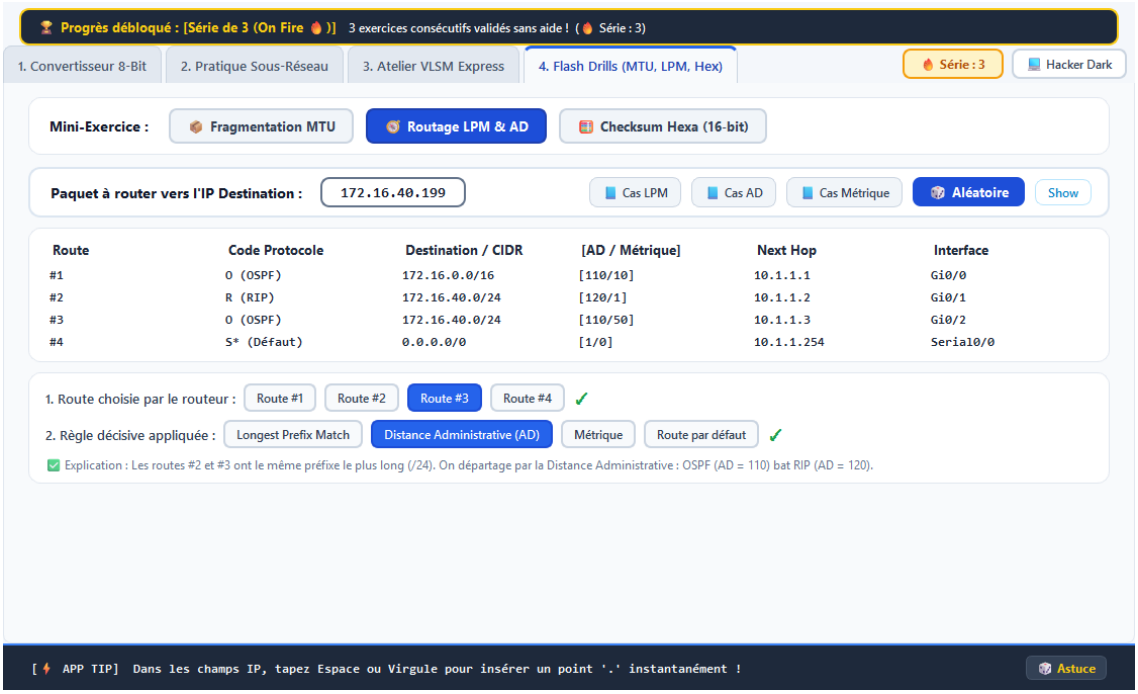



Figure 11.1: Onglet 4 — Résolution d'un exercice de routage LPM avec explications pédagogiques pas à pas.

## 11.2 Algorithme de Vérification LPM

```
def check_lpm_answer(self):
    selected_row = self.get_selected_route_row()
    selected_rule = self.combo_rule.currentText()
    problem = self.current_lpm_problem

    is_row_ok = (selected_row == problem.winning_row_id)
    is_rule_ok = (selected_rule == problem.decisive_rule)

    if is_row_ok and is_rule_ok:
        self.feedback_label.setText(f"Parfait ! {problem.explanation}")
        self.feedback_label.setProperty("status", "success")
        self.streak_mgr.record_win(without_show=not self.used_show, category="
            lpm_clean")
    else:
        self.feedback_label.setText("Incorrect. Verifiez la regle de priorite : LPM >
             AD > Metrique.")
        self.feedback_label.setProperty("status", "error")
```

# Part VI

# Système, Packaging & Blueprint de Portage Java OOP

# Chapter 12

# Cycle de Vie de l'Application & Compilation Standalone

## 12.1 Point d'Entrée `converter.py`

Le cycle de vie d'une application PyQt6 s'articule en 4 étapes rigoureuses :

```
import sys
from PyQt6.QtWidgets import QApplication
from ui.main_window import MainWindow

def main():
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()
```

L'appel `app.exec()` démarre la boucle d'événements principale et bloque jusqu'à la fermeture de la fenêtre. `sys.exit()` relaie alors le code de sortie entier à l'OS.

## 12.2 Compilation Binaire Standalone avec PyInstaller

L'exécutable portable autonome (`converter.exe`, 38 Mo) est généré en encapsulant l'interpréteur Python, PyQt6 et les scripts sans dépendance externe :

```
pyinstaller --onefile --noconsole --name "converter" converter.py
```

# Chapter 13

# Blueprint de Portage vers Java 17/21 Orienté Objet

## 13.1 Correspondance des Modèles de Données (`record` Java)

Toutes les structures `@dataclass(frozen=True)` de `engine/` se transposent sans aucune modification algorithmique en `record` Java :

```
package ma.inpt.aseds.subnet.engine;

public record OctetAnalysis(
    int value,
    String binaryStr,
    String hexStr,
    boolean isValidMaskOctet,
    int onesComplement,
    String onesComplementBinary,
    int blockSize
) {}
```

## 13.2 Pièges de l'Arithmétique 32 bits en Java (`int` signé vs `long`)

En Java, le type primitif `int` est signé sur 32 bits (de $-2^{31}$ à $2^{31}-1$). Ainsi, `0xFFFFFFFF` vaut $-1$.

1. Utiliser le type **`long` (64 bits)** ou l'opérateur de décalage logique non signé `>>>` pour les adresses 32 bits :

```
long maskInt = (cidr > 0) ? ((0xFFFFFFFFL << (32 - cidr)) & 0xFFFFFFFFL) : 0L;
long wildcardInt = (~maskInt) & 0xFFFFFFFFL;
long networkInt = ipInt & maskInt;
long hostIdInt = ipInt & wildcardInt;
long broadcastInt = networkInt | wildcardInt;
```

2. Pour le Checksum 16 bits :

```
int carried = rawSum;
while (carried > 0xFFFF) {
    carried = (carried & 0xFFFF) + (carried >>> 16);
}
int checksum = (~carried) & 0xFFFF;
```

## 13.3 Architecture de Packages Java Recommandée

- `ma.inpt.aseds.subnet.engine` : 100 % Java standard, couvert par des tests unitaires **JUnit 5**.
- `ma.inpt.aseds.subnet.ui` : Vues d'interface développées en JavaFX 21 ou Swing FlatLaf reprenant les mêmes feuilles de style CSS/QSS.

# Conclusion & Synthèse Pédagogique

Le présent manuel compile et formalise l'intégralité des connaissances pédagogiques et d'ingénierie logicielle intégrées dans la suite d'apprentissage **IPv4 Subnet Toolkit v1.2.0**.

En combinant une compréhension mathématique rigoureuse de l'algèbre binaire IPv4, le respect strict de la Clean Architecture et des principes d'ergonomie cognitive modernes, ce projet constitue une référence complète pour les étudiants et ingénieurs de l'INPT dans leur maîtrise des réseaux informatiques et de la conception logicielle robuste.